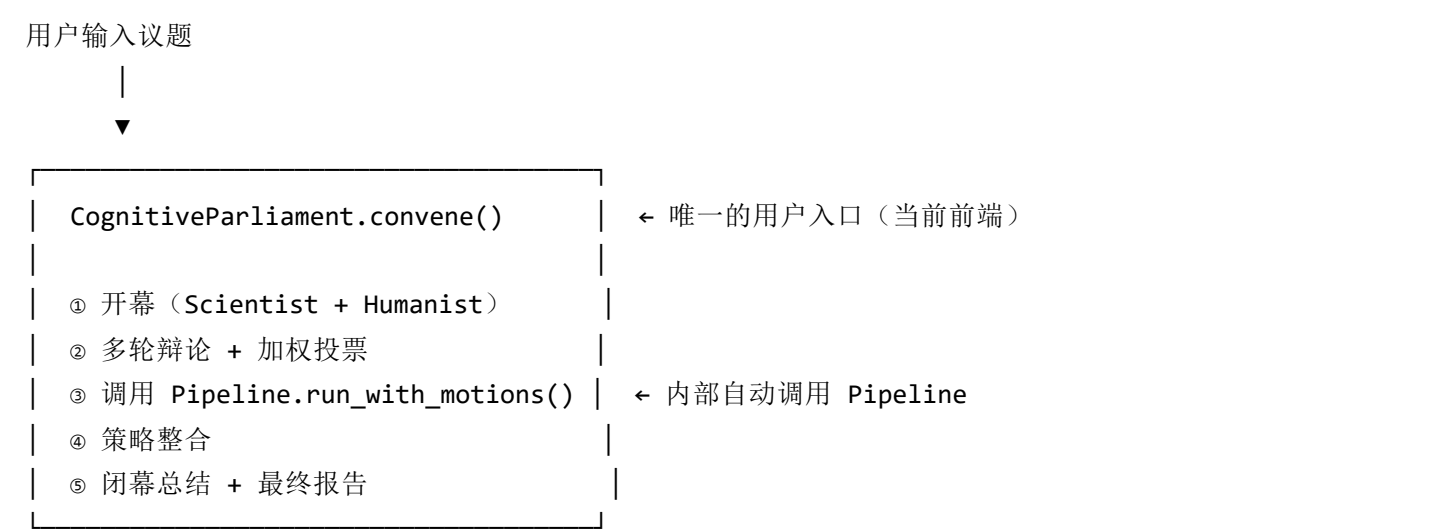


云观星传 — 智能体调用关系与完整流程

本文档精确描述 Pipeline 与 Parliament 两种模式的调用关系、它们在代码中如何互调、以及完整的执行路径。

一、正确认知：Parliament 包裹 Pipeline

之前可能认为"系统有两套并列的协作模式，用户选择用哪个"。实际代码中，当前前端只调用 **Parliament**，**Parliament** 内部再调用 **Pipeline**。



Pipeline 的独立入口 `POST /api/analyze/run` 仍然保留可用，但前端 TaskCenter 目前只走 Parliament 路径。

二、完整调用链路（从前端到 LLM）

2.1 前端入口

文件: `frontend/src/pages/TaskCenter.tsx` (行 69-93)

- 用户输入"嫦娥六号" → 点击"启动认知议会"
- `conveneParliament("嫦娥六号", maxRounds=5, maxPipelineRounds=3)`
 - `POST /api/parliament/convene`
 - 返回 `{ task_id: "parl_20260725..." }`
 - 前端每2秒轮询 `GET /api/parliament/status/{task_id}`
 - 完成后跳转到 `/parliament` 页面展示结果

关键代码 (TaskCenter.tsx:73):

```
const { task_id } = await conveneParliament(t, maxRounds, maxPipelineRounds)
```

前端只有这一个入口——没有同时调用 Pipeline 的地方。

2.2 后端 Parliament 路由

文件: `api/routes/parliament.py` (行 223-263)

```
def run_parliament_task(task_id, topic, max_rounds, max_pipeline_rounds):
    parliament = CognitiveParliament(
        max_rounds=max_rounds,                # 辩论轮次（默认5）
        max_pipeline_rounds=max_pipeline_rounds,    # Pipeline评测轮次（默认3）
        progress_callback=make_parliament_callback(task_id),
    )
    transcript = parliament.convene(topic=topic, science_facts=science_facts)
```

2.3 CognitiveParliament.convene() 内部执行顺序

文件: `src/pipeline.py` (行 744-850)

步骤	操作	涉及哪些 Agent	代码位置
①	开幕	Scientist + Humanist 做开场报告	pipeline.py:766-773
	<code>debate_engine.open_parliament()</code>	Scientist 生成科学动议 (最多3条)	debate_engine.py:97-126
		Humanist 生成文化动议 (最多2条)	debate_engine.py:129-159

步骤	操作	涉及哪些 Agent	代码位置
②	辩论循环	Speaker 主持，所有 Agent 参与	pipeline.py:777-792
	<code>while not should_close():</code>	每轮：Speaker规划 → 联网搜索 → 发言 → 投票	debate_engine.py:188-348
	<code>debate_engine.debate_round()</code>	循环直到 <code>should_close()</code> 返回 True	
③	Pipeline 接驳	Pipeline 对通过的动议做校验 +策略+评测	pipeline.py:794-828
	<code>Pipeline.run_with_motions()</code>	跳过前3步， 直接从校验层开始	pipeline.py:291-454
④	策略整合	StrategyAgent 融合辩论 + Pipeline 结果	pipeline.py:821-828
⑤	闭幕	Speaker 生成闭幕总结	pipeline.py:832-834
⑥	最终报告	LLM 生成结构化总结报告 (核心结论/TOP3策略/ 风险/受众建议)	pipeline.py:838-843

三、辩论循环详解

文件: src/parliament/debate_engine.py

3.1 单轮辩论 (debate_round() , 行 188-348)

Speaker.plan_round()
→ 决定本轮讨论主题 (current_topic)
→ 决定谁发言 (next_speakers, 2-4人)
→ 决定投票权重 (speaker_weights)
→ 决定表决哪个动议 (motion_to_vote)
联网搜索 (Tavily + 百炼 WebSearch)
→ 每轮一次, 所有发言者共享
顺序发言 (for speaker_name in next_speakers)
agent.run({
task_type: "debate_speech",
current_motion: ...,
previous_speeches: [...],
})
→ 返回 { stance, content, references }
→ 相似度检查 (bigram重叠>0.8标记为"重述")
投票表决 vote_on_motion()
每个Agent独立投票 → 加权计算 → 判定结果
→ 僵持→Speaker裁定
→ 记录少数派意见

3.2 Agent 辩论中的角色转换

同一个 Agent 类在 Pipeline 和辩论中扮演不同角色：

Agent 类	Pipeline 角色	辩论角色	发言风格
ScienceAgent	科学事实提取者	Scientist 科学专家	从证据角度评价动议
HypothesisAgent	假设生成者	Skeptic 质疑者	强制找漏洞（至少1个）， 温度提高到0.5
HumanistAgent	（不参与Pipeline）	Humanist 人文学者	文化风险审查，温度提高到0.5

Agent 类	Pipeline 角色	辩论角色	发言风格
StrategyAgent	策略转译师	Strategist 策略师	可操作性评估
EvaluatorAgent	评测员	Evaluator 评估者	五维标准独立评估
ContextAgent	语境分析师	不参与辩论	—

3.3 动态权重

Speaker 根据辩论主题自动切换投票权重（`speaker.py:18-35`）：

讨论类型	Scientist	Skeptic	Humanist	Strategist	Evaluator
事实讨论	0.40	0.25	0.10	0.10	0.15
文化适配	0.10	0.15	0.40	0.20	0.15
策略可行性	0.10	0.15	0.15	0.35	0.25
方法论	0.25	0.35	0.10	0.10	0.20

谁的权重高，谁在投票时的话语权就大。

3.4 闭幕条件 (`should_close()` , 行 500-516)

满足任一即闭幕：

- 辩论轮次 ≥ 5 轮
- 所有动议已表决完毕
- 连续 2 轮评分无提升

四、Parliament 如何调用 Pipeline

最关键的一段代码在 `CognitiveParliament.convene()` 第 794-828 行：

```

# 筛选通过的动议
passed_motions = [
    m for m in self.debate_engine.motions
    if any(v.motion_id == m.motion_id and v.result == "passed"
           for v in self.debate_engine.votes)
]

# 调用 Pipeline（跳过前3步，直接从校验开始）
pipeline = Pipeline(
    progress_callback=self.progress_callback,
    max_iterations=self.max_pipeline_rounds
)
pipeline_result = pipeline.run_with_motions(
    topic=topic,
    motions=[m.model_dump() for m in passed_motions],
    minority_opinions=self.debate_engine.minority_opinions,
    debate_transcript=[r.model_dump() for r in self.debate_engine.rounds],
    science_facts=science_facts,
    context_analysis=context_analysis,
)

```

run_with_motions() 做了什么 （ pipeline.py:291-454 ）

议会通过的动议



动议 → 假设格式转换

motion.content → hypothesis.statement

motion.evidence → hypothesis.evidence_chain

少数派意见 → hypothesis.falsification_criteria

联网搜索（Tavily + 百炼 WebSearch）

交叉校验

CrossValidator 四路投票（RAG+KG+Wikidata+Wikipedia）

策略+评测+迭代（循环，最多 max_pipeline_rounds 轮）

for round in range(max_pipeline_rounds):

StrategyAgent → 生成策略

EvaluatorAgent → 五维评分+受众模拟

→ 加权总分 ≥ 75 → 通过 ✓

→ 未通过 → 生成迭代反馈 → 下一轮

迭代的额外输入：

- minority_opinions（少数派意见）

- debate_transcript_summary（辩论摘要）



返回 { verification_results, strategies, evaluation, search_sources }

随后这些结果被传回 CognitiveParliament ，用于整合策略和生成最终报告。

run_with_motions() 与 run() 的对比

对比维度	Pipeline.run()	Pipeline.run_with_motions()
谁来调用	API /analyze/run （独立使用）	CognitiveParliament.convene()
Step 0: 搜索	✓	✓

对比维度	Pipeline.run()	Pipeline.run_with_motions()
Step 1: 科学理解	✓	✗ 跳过（动议已替代假设）
Step 2: 语境分析	✓	✗ 跳过
Step 3: 假设生成	✓	✗ 跳过（动议 = 假设）
Step 4: 校验	✓	✓
Step 5-7: 策略+评测+迭代	✓	✓
少数派意见注入	无	✓
辩论摘要注入	无	✓

五、完整端到端执行序列（一次"嫦娥六号"任务）

用户输入 "嫦娥六号"

└─ [前端] TaskCenter → POST /api/parliament/convene

└─ [后端] CognitiveParliament.convene("嫦娥六号")

```
|| 阶段一：开幕（~2次 LLM调用） ||
|| Scientist.run(task_type="opening_report") ||
||   → 生成 3 条科学动议 ||
|| Humanist.run(task_type="opening_report") ||
||   → 生成 2 条文化动议 ||

|| 阶段二：辩论循环（每轮~8次 LLM调用） ||
|| 第1轮： ||
||   Speaker.plan_round() → 事实讨论 ||
||   联网搜索（1次） ||
||   Scientist 发言（stance: support） ||
||   Skeptic 发言（stance: oppose, 强制找漏洞） ||
||   投票 M_S001（5个Agent各1次） ||
||   → yes=0.55 > no=0.30 → passed ✓ ||
|| 第2轮： ||
||   Speaker.plan_round() → 文化适配 ||
||   Humanist 发言（stance: amend） ||
||   Strategist 发言（stance: support） ||
||   投票 M_H001: 僵持 → Speaker裁定 ||
||   → 附条件通过（amended） ||
|| ...最多5轮，直至 should_close() ||

|| 阶段三：Pipeline 接驳（每轮~2次+校验） ||
|| 筛选通过的动议（M_S001, M_H001...） ||
```

```

|| run_with_motions():
||   → 联网搜索 (Tavily + 百炼 双引擎)
||   → 四路交叉校验 (每个断言4条路径)
||
|| 第1轮策略:
||   StrategyAgent → 3条传播策略
||   EvaluatorAgent → 加权总分 68
||   X 未达75分 → 迭代
||
|| 第2轮策略:
||   StrategyAgent → 改进后策略 (注入少数派意见)
||   EvaluatorAgent → 加权总分 82
||   ✓ 通过!

```

```

|| 阶段四: 收尾 (~3次 LLM调用)
||
|| StrategyAgent → 融合辩论+Pipeline的最终策略
|| Speaker.close_parliament() → 闭幕总结
|| LLM → 结构化最终报告
|| {
||   one_line_takeaway: "一句话结论",
||   core_conclusion: "150-250字核心结论",
||   top_strategies: [TOP3策略],
||   risk_warnings: [风险提示],
||   audience_recommendations: [受众建议]
|| }

```

— [前端] 轮询检测到 completed

```

→ GET /parliament/result/{task_id}
→ 跳转 /parliament → 展示辩论记录+策略+校验报告

```


七、为什么这样设计

辩论层与校验层的分离

辩论中 Agent 说"法国媒体对嫦娥六号的报道以竞争框架为主"——这只是一个**观点**。Parliament 负责多角度讨论一个观点是否合理，Pipeline 负责验证这个观点的**事实基础**。

少数派意见不丢失

辩论中被多数票否决的观点，在 `run_with_motions()` 中作为 `minority_opinions` 注入。这意味着即使某个观点被投票否决，它仍然影响后续策略生成——通常体现在"风险提示"中。

分工角色清晰

层	职责	谁做
辩论层 (Parliament)	提出观点、质疑观点、达成共识	6个 Agent + 1个 Speaker
校验层 (Pipeline → CrossValidator)	验证事实真伪	RAG + KG + Wikidata + Wikipedia
策略层 (Pipeline → StrategyAgent)	生成可执行传播策略	StrategyAgent + EvaluatorAgent 迭代

八、一次完整运行的 LLM 调用次数估算

以默认参数（5 轮辩论 + 3 轮 Pipeline 评测）为例：

阶段	调用次数	明细
开幕	2	Scientist + Humanist 各1次
辩论（每轮）	7-9	Speaker 1次 + 2-4个发言者 + 5个投票
辩论总计（按5轮）	35-45	
Pipeline 校验	4-6	取决于断言数量

阶段	调用次数	明细
Pipeline 策略+评测（每轮）	2	StrategyAgent + EvaluatorAgent
Pipeline 总计（按2轮通过）	6-10	
策略整合	1	
闭幕+最终报告	2	Speaker总结 + LLM结构化报告
总计	约 46-64 次 LLM调用	

这是当前架构最大的成本点。优化方向见下文。

九、经验池的当前覆盖范围

Pipeline 评测每轮完成后：

```
EvaluationEngine.log_experience()

- └ 写入内存 pool（会话级）
- └ 写入 SQLite experience_store
  - └ experiences 表：  五维评分 + 低分维度 + 反馈
  - └ topic_embeddings 表：议题embedding（相似议题检索）

```

下次处理相似议题时：

```
EvaluationEngine.load_past_experience(topic)

- └ 相似议题检索（embedding 余弦相似度）
- └ 常见弱点统计
- └ 全局改进趋势

```

当前局限性：经验池只记录 Pipeline 评测层的经验（评分数据），**不记录辩论层的经验**。比如"哪种权重模板效果好""哪个 Agent 组合发言质量高"这类辩论策略的经验目前没有被积累。

最后更新：2026-07-25